

A Csound-Based Programmable Sound Engine for Machine-Listening-Driven Live Electronics

Liwei Shi¹ and Chen Bai²

¹ Shanghai Conservatory of Music
lruma129@yeah.net

² Communication University of Zhejiang
baichen1985@163.com

Abstract. This paper presents a programmable Csound sound engine for machine-listening-driven real-time audio processing in live electronics. The system integrates Max/MSP, FluCoMa, a multilayer perceptron (MLP) regression model, and Csound in a layered architecture for audio analysis, parameter mapping, and sound processing. Max/MSP handles audio input, control scheduling, and OSC/UDP communication, while FluCoMa extracts acoustic features. The MLP maps these features to two continuous performance-state coordinates, which a subsequent mapping layer converts into eight digital signal processing (DSP) control values. Training takes place before performance. Network parameters and feature-standardization parameters are saved in separate JSON files and loaded for inference during performance. Csound applies delay, spectral processing, nonlinear transformation, random modulation, and spatial diffusion to the input audio according to the received control values. The paper describes data transfer, parameter mapping, and smoothing across the layers and examines the processing workflow through model-driven offline replay of a complete recording. The resulting design provides a programmable connection between machine listening and real-time electronic sound processing.

Keywords: Csound; machine listening; parameter mapping; FluCoMa; MLP

1 Introduction

Machine listening allows a performer’s sound to provide both audio input and control information. Supervised learning connects acoustic features to musician-defined targets [1]. This paper connects Max/MSP and FluCoMa analysis [2] to a two-output multilayer perceptron (MLP) and a programmable Csound engine. Deterministic mapping translates the learned state into eight coupled DSP controls. The contribution is an explicit account of training, JSON storage, mapping, smoothing, and processing, supported by model-driven offline replay. Here, live electronics denotes electronic processing of live instrumental sound; the model maps performance states rather than composing or generating audio autonomously.

2 Architecture, Training, and Mapping

2.1 Audio and Control Paths

The proposed live setup has Max/MSP and a separate Csound process reading one input channel through the same Core Audio interface. Max analyzes audio and predicts controls; Csound processes audio and produces stereo output (Fig. 1). Csound receives OSC using OSCinit and OSClisten [3]. Max sends eight floats in Table 1’s order through `prepend /cs/params` and `udpsend 127.0.0.1 7770` [4]. The receiver uses the type signature `fffffff`. At nominal 20 ms intervals, `qmetro 20` maps the latest cached B/R values; messages have no inference-frame timestamp. Shared-device operation and actual timing remain to be measured.

2.2 Features, Targets, and JSON Files

The input is $\mathbf{x} = [c_1, \dots, c_{13}, L, P]$: thirteen MFCCs, loudness, and true peak. The first audio channel is analyzed with `fluid.mfcc~ 13 @startcoeff 1` and `fluid.loudness~`. FluCoMa 1.0.9 settings are 1024-sample windows/FFT and a 512-sample hop. Loudness updates concatenate the

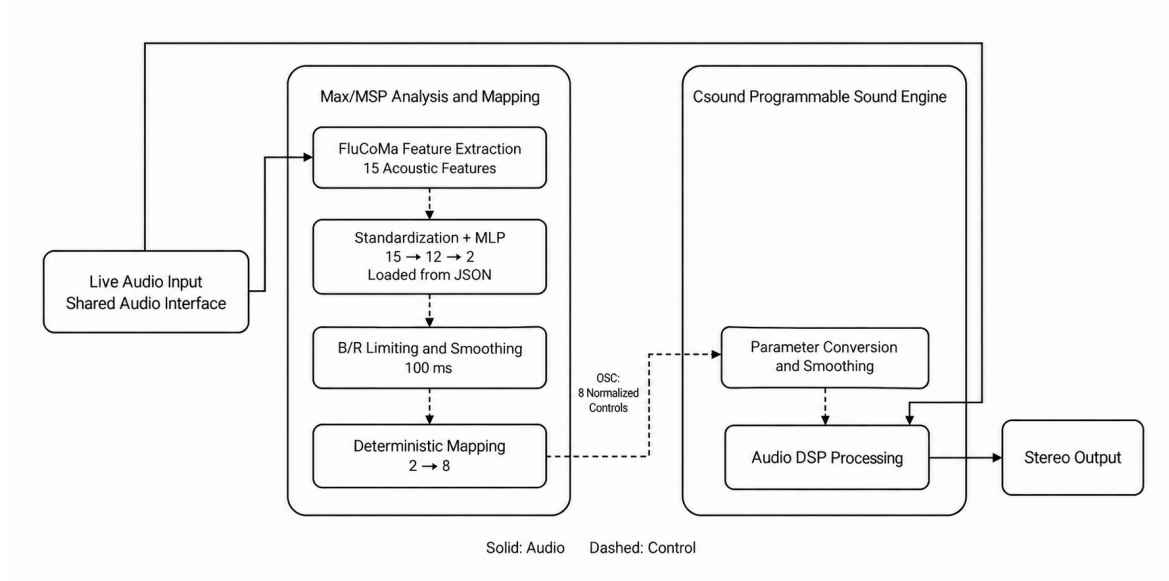


Fig. 1. Proposed live configuration. Solid lines carry audio; dashed lines carry control.

latest MFCC and loudness lists. Standardization applies $z_j = (x_j - \mu_j)/\sigma_j$ using fifteen stored means and standard deviations; feature definitions and order remain unchanged at inference.

Training material comprises twelve 48 kHz, 24-bit stereo cello recordings from *Trill III*, totaling 138.52 s. Six use sul ponticello (SP, near the bridge; $B = +1$), and six sul tasto (ST, over the fingerboard; $B = -1$). In each group, files 1–4 carry the normal bow-pressure label $R = 0$ and files 5–6 the high-pressure label $R = 1$. These are author-defined targets, not physical measurements or class probabilities.

Preparation pairs feature and target datasets, standardizes features with `fittransform`, trains with `fit`, and saves separate network and standardization JSON files [5]. The network is $15 \rightarrow 12 \rightarrow 2$, with sigmoid hidden units and identity outputs [6]. Saved settings are `batchsize 64`, `learnrate 0.01`, `momentum 0.9`, `maxiter 3000`, and `validation 0.1`. This internal point-level validation follows standardization; it is not an independent recording-level test. Historical fitting-frame counts and error logs are unavailable.

Before performance, `TrillIII_scaler.json` and `TrillIII_BR_network.json` are loaded; `transformpoint` standardizes features and `predictpoint` returns B/R. Weights remain fixed; retraining belongs to preparation.

2.3 Two Learned Outputs and Eight Controls

After clipping and smoothing $B \in [-1, 1]$ and $R \in [0, 1]$, Max defines $b = (B+1)/2$ and $r = R$. Table 1 specifies the normalized OSC values and Csound conversions. The rules in `br_to_eight.js` give eight coupled controls with only two independent dimensions. Changing these rules does not require retraining; independently learning eight targets would require new labels and a different model.

3 Csound Implementation

3.1 Choice of Engine and Process

Csound’s textual orchestra makes signal flow and computation rates explicit [7]. It allows DSP inspection and modification independently of the learning patch; the effects themselves are also available in MSP. A separate process supports tests with fixed or recorded controls. Embedding via `csound~` or `csound6~` is a valid alternative that simplifies host routing [8]. The chosen separation adds audio-routing and message-scheduling work; no lower latency, reduced jitter, or superior sound quality is claimed.

Table 1. Two-to-eight mapping. *pitch* denotes frequency shift, not semitone transposition.

Control / role	Normalized value	Csound conversion
density: modulation rate	r	$(0.2 + 11.8r)$ Hz
time: delay	$1 - r$	$20 + 980(1 - r)$ ms
position: pan target	b	b (0–1)
pitch: frequency shift	b	$-200 + 400b$ Hz
spectral: nonlinear drive	$(b + r)/2$	$1 + 7(b + r)/2$
brightness: low-pass cutoff	b	200×40^b Hz
space: pan separation	$1 - b$	$1 - b$ (0–1)
instability: modulation depth	r	$0.15r$

3.2 Smoothing and Processing

Max clips B/R and applies retargetable 100 ms ramps using `pack f 100` and `line 0`. [9]. The mapping rejects non-finite inputs. Csound further smooths seven controls with `portk` half-times of 0.02 s; delay uses 0.05 s [10]. Brightness is smoothed before exponential conversion. Delay time is converted to seconds, smoothed, interpolated with `interp`, and sent to `deltap3` [11]. Its moving read head can produce pitch glides or transients. Ramp and half-time settings are not measurements of system latency.

The engine runs at 48 kHz, `ksmps=32`, stereo output, and `0dbfs=1`. Table 2 follows the wet-signal order; ranges are in Table 1. Feedback remains in the linear delay loop. Waveshaping normalization does not ensure equal loudness, and pan clipping limits actual separation. Final output mixes dry and wet audio with gains of 0.5 each.

Table 2. Wet-chain opcodes and fixed settings. p is pan position; w is nominal separation.

Module	Csound opcodes	Fixed settings
Delay	<code>delayr</code> , <code>deltap3</code> , <code>delayw</code>	1.2 s buffer; feedback 0.25; original/delayed gains 0.5 each
Spectral	<code>hilbert</code> , <code>quadrature</code> <code>oscili</code> , <code>tone</code>	Frequency shift, then low-pass filter
Nonlinearity	<code>tanh</code>	Drive d : 1–8; $y = \frac{\tanh(dx)}{\tanh(d)}$
Modulation	<code>randomi</code>	Random values in $[-1, 1]$
Stereo diffusion	<code>delay</code> , <code>pan2</code>	7/11 ms delays; $p \pm w/2$ clipped to $[0, 1]$; two outputs summed at 0.5 each

For example, brightness conversion uses the sixth received control and its initial value `ib`:

```
kBrightness portk kU[5], 0.02, ib
kCutoff = 200 * pow(40, kBrightness)
aFiltered tone aShifted, kCutoff
```

4 Implementation Verification

A 101×101 grid checked 10,201 B/R pairs; all mappings were finite and matched Table 1’s order and ranges. Csound 6.18 rendered the first 2 s of `SP_1` at the four label endpoints and $(B, R) = (0, 0.5)$. All five stereo outputs were finite, with no absolute sample amplitude above 1.

The complete dry recording of *Trill III* was analyzed offline in Max 9.0.9/FluCoMa 1.0.9 using `fluid.bufmfcc~` and `fluid.bufloudness~`. First-channel analysis used the stated window/hop settings and padding 1; matching feature-frame indices were combined. The saved standardizer and MLP produced the raw predictions. The control branch was clipped, initialized from the first clipped frame, and reconstructed offline with 100 ms target ramps sampled every 20 ms. All eight normalized controls were finite and within $[0, 1]$.

Figure 2 shows raw predictions and reconstructed controls; Table 3 summarizes the replay. Parameter files drove full-recording Csound processing with Table 1’s conversions and the stated smoothing. This is model-driven offline replay, without real-time OSC, shared-device capture, or end-to-end latency measurement. Time-aligned B/R targets are absent and independence from training material is unverified; the results establish processing operation, not prediction accuracy or generalization.

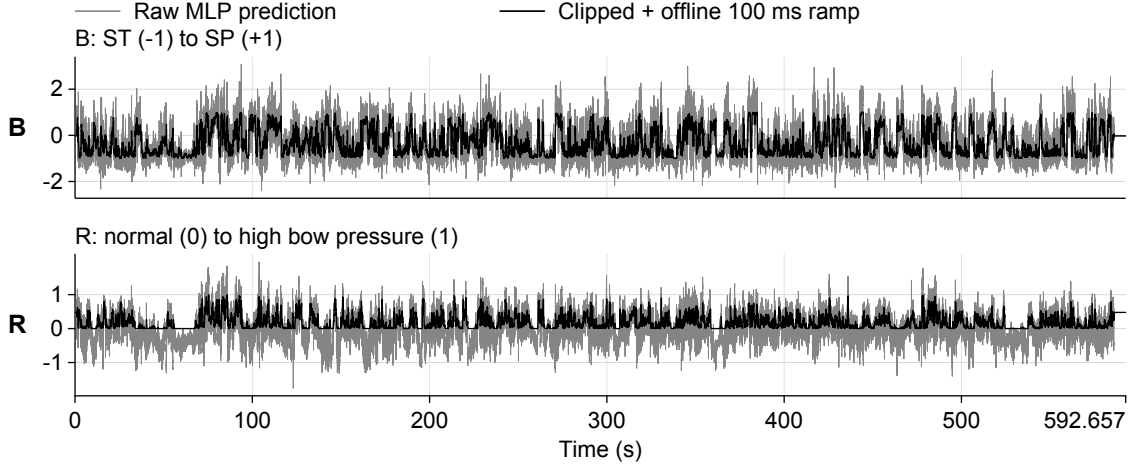


Fig. 2. Full-recording raw predictions (gray) and clipped, offline reconstructed controls (black). All prediction and control points are plotted; the traces are not a target-versus-prediction accuracy comparison.

Table 3. Model-driven offline replay results. RMS pools both output channels; frame counts refer to this replay.

Measure	Result
Input recording duration	592.657 s
Feature and raw prediction frames	55,562
Eight-parameter control records	29,633
Raw B range	−2.398 to 3.077
Raw R range	−1.758 to 1.964
Output sample peak (0 dBFS = 1)	0.488175
Output RMS	0.027844
Non-finite output samples	0
Output samples with absolute amplitude > 1	0

5 Conclusion

The engine links fifteen features, two learned coordinates, and eight explicitly coupled Csound controls. Separate JSON files distinguish preparation from fixed-model inference; smoothing is distributed between Max and Csound. Mapping checks and full-recording offline replay support the implemented workflow. Independent labeled recordings, real-time measurements, and listening evaluation remain necessary to assess generalization, response, and musical outcomes.

References

1. Fiebrink, R. A., Caramiaux, B.: The machine learning algorithm as creative musical tool. In: Dean, R. T., McLean, A. (eds.) *The Oxford Handbook of Algorithmic Music*, pp. 181–208. Oxford University Press (2018). <https://doi.org/10.1093/oxfordhb/9780190226992.013.23>
2. Tremblay, P. A., Roma, G., Green, O.: Enabling programmatic data mining as musicking: The Fluid Corpus Manipulation Toolkit. *Computer Music Journal* 45(2), 9–23 (2021). https://doi.org/10.1162/comj_a_00600
3. Csound Community: The Canonical Csound Reference Manual. Version 6.18.0. <https://csound.com/docs/manual/index.html>
4. Cycling '74: udpsend. <https://docs.cycling74.com/reference/udpsend/>
5. Moore, T.: Neural Network Training. Learn FluCoMa. <https://learn.flucoma.org/learn/mlp-training/>
6. Moore, T.: Neural Network Parameters. Learn FluCoMa. <https://learn.flucoma.org/learn/mlp-parameters/>
7. Boulanger, R. (ed.): *The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming*. MIT Press (2000).
8. Duncan, I. C. T.: Csound6~ (csound_max) [software]. GitHub. https://github.com/iainctduncan/csound_max
9. Cycling '74: line. <https://docs.cycling74.com/reference/line/>
10. Csound Community: portk. <https://csound.com/docs/manual/portk.html>
11. Csound Community: deltap3. <https://csound.com/docs/manual/deltap3.html>